

Understanding Tree DFS Recursion

Yes — this is **one of the biggest confusions in Tree + DFS recursion**. The main problem is usually:

"I called DFS... now where does the answer come back?"
 "Should I update before the recursive call, after it, or inside it?"
 "What should DFS return?"

Let's build one **complete mental model** for this.

1. First understand: Every DFS function has a job

Before writing recursion, ask:

What information does my parent need from me?

This decides what your `dfs()` should **return**.

For example:

Case A: Parent needs height

```
C++
int dfs(TreeNode* node)
```

DFS returns the height of the subtree.

Case B: Parent needs sum

```
C++
int dfs(TreeNode* node)
```

DFS returns the sum of the subtree.

Case C: Parent doesn't need anything back

```
C++
void dfs(TreeNode* node)
```

Then you usually update a global/reference variable.

2. The most important recursion structure

Almost every binary tree DFS looks like this:

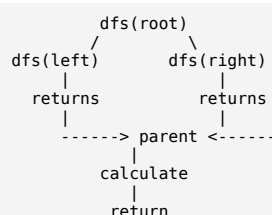
```
C++
ReturnType dfs(TreeNode* node) {
    // 1. Base case
    if (node == NULL)
        return BASE_VALUE;

    // 2. Recursive calls
    auto left = dfs(node->left);
    auto right = dfs(node->right);

    // 3. Use returned values
    // calculate something

    // 4. Return result to parent
    return answer;
}
```

Think of it like this:



The **child solves its subtree first**, then returns information to its parent.

3. Example: Maximum Depth of Binary Tree

Let's take:



We want:

Maximum Depth = 3

Question:

What does each node need from its children?

It needs:

"Tell me the maximum depth of your subtree."

So DFS should return an `int`.

```

C++
int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    return 1 + max(left, right);
}
  
```

How recursion actually flows

Start:

dfs(1)

It cannot calculate yet because it needs:

dfs(2)
dfs(3)

So first:

dfs(2)

Again, 2 needs its children.

dfs(4)
dfs(NULL)

Then:

dfs(4)

4 has no children:

```

left = 0
right = 0

return 1
  
```

So 4 says to 2:

"My subtree height is 1."

Now at node 2:

```

left = 1
right = 0

return 1 + max(1, 0)
  
```

```
return 2
```

Now 2 says to 1:

"My subtree height is 2."

Then node 3:

```
return 1
```

Finally node 1:

```
left = 2
right = 1
return 1 + max(2, 1)
return 3
```

Flow:

```

      1
     / \
    2   3
   /
  4

dfs(4) returns 1
  ↓
dfs(2) receives 1
dfs(2) returns 2
  ↓
dfs(1) receives 2

dfs(3) returns 1
  ↓
dfs(1) receives 1

dfs(1):
1 + max(2, 1) = 3

```

4. When do we UPDATE a value?

This depends on **what type of answer we are calculating**.

There are mainly **3 important patterns**.

Pattern 1: Return answer to parent

Use this when:

Each node calculates an answer using its children's answers.

Example:

- Maximum depth
- Minimum depth
- Balanced Binary Tree
- Diameter helper
- Subtree sum

Template:

```
C++
int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    int answer = /* use left and right */;

    return answer;
}
```

Mental model:

```

Child solves
  ↓
Child returns result
  ↓

```

```

Parent receives result
↓
Parent combines results
↓
Parent returns its result

```

Pattern 2: Update a global answer

Sometimes the answer is calculated **at a node**, but the parent doesn't need that exact answer.

Example: **Diameter of Binary Tree**.

At every node:

```
diameter through node = leftHeight + rightHeight
```

We want the maximum diameter anywhere.

```

C++
int diameter = 0;

int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    diameter = max(diameter, left + right);

    return 1 + max(left, right);
}

```

Here there are **two different things**.

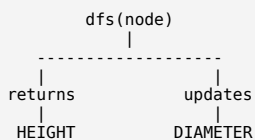
What DFS returns:

```
height
```

What global variable stores:

```
maximum diameter
```

This is extremely important.



The parent needs the **height**, not the diameter.

But we calculate the diameter while we have:

```

C++
left
right

```

So:

```

C++
diameter = max(diameter, left + right);

```

Then return:

```

C++
return 1 + max(left, right);

```

Pattern 3: Pass information from parent to child

Sometimes the information flows **downward**, not upward.

Example:

Maximum Difference Between Node and Ancestor

We can pass:

```
minimum ancestor
maximum ancestor
```

to children.

```
C++
void dfs(TreeNode* node, int mini, int maxi) {
    if(node == NULL)
        return;

    mini = min(mini, node->val);
    maxi = max(maxi, node->val);

    dfs(node->left, mini, maxi);
    dfs(node->right, mini, maxi);
}
```

Here DFS doesn't need to return anything.

Why?

Because the information is flowing like:

```
Parent
  ↓
Pass information
  ↓
Child
  ↓
Updated information
  ↓
Grandchild
```

This is called a **top-down DFS style**.

5. The BIGGEST concept: Top-Down vs Bottom-Up

This will solve a lot of your confusion.

A. Top-Down DFS

Information goes:

```
ROOT
  ↓
CHILD
  ↓
GRANDCHILD
```

You pass something as a parameter.

Example:

```
C++
void dfs(node, currentSum)
```

At node:

```
C++
currentSum += node->val;
```

Then pass it:

```
C++
dfs(node->left, currentSum);
dfs(node->right, currentSum);
```

Example: Path Sum

```
C++
void dfs(TreeNode* node, int sum) {
    if(node == NULL)
        return;

    sum += node->val;

    dfs(node->left, sum);
```

```
dfs(node->right, sum);
}
```

Information flows downward.

```

      5
     / \
    4   8

sum = 0
dfs(5, 0)
  ↓
sum = 5
dfs(4, 5)
  ↓
sum = 9
dfs(8, 5)
  ↓
sum = 13
```

Each child receives information from the parent.

B. Bottom-Up DFS

Information goes:

```

LEAF
↑
CHILD
↑
ROOT
```

Example: Height.

```
C++
int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    return 1 + max(left, right);
}
```

Here:

```

leaf returns answer
  ↑
parent receives answer
  ↑
root receives answer
```

6. The easiest way to decide what DFS should return

Before coding, write this sentence:

"For node X to solve its problem, what information does it need from its children?"

Let's see examples.

Maximum Depth

Node needs:

Maximum depth from left and right.

So:

```
C++
dfs() returns height
```

Balanced Binary Tree

Node needs:

Height of left subtree and height of right subtree.

So:

```
C++
dfs() returns height
```

But also update/check:

```
abs(left - right) <= 1
```

Diameter

Node needs:

Height from left and right.

So:

```
C++
dfs() returns height
```

Global:

```
C++
maxDiameter
```

is updated.

Maximum Path Sum

Node needs:

Maximum gain from left and right.

So:

```
C++
dfs() returns maximum gain
```

But global answer:

```
C++
maxPath
```

is updated.

```
dfs returns:
maximum path going UP

global stores:
maximum path anywhere
```

This distinction is very important.

7. A universal question checklist

Whenever you see a Tree DFS question, ask these **5 questions**.

Question 1: What does a node need?

For example:

```
Height?
Sum?
Maximum?
Minimum?
Path information?
```

That often tells you what DFS returns.

Question 2: Is information going DOWN or UP?

Down:

```
C++
dfs(node, information)
```

Example:

```
C++
dfs(node, currentSum)
dfs(node, minValue)
dfs(node, maxValue)
```

Up:

```
C++
int left = dfs(node->left);
int right = dfs(node->right);
```

Example:

```
height
diameter helper
subtree sum
maximum gain
```

Question 3: Do I need a global answer?

Ask:

"Is the answer at every node a candidate for the final answer?"

Example:

```
Diameter
Maximum Path Sum
Maximum Difference
```

Then usually:

```
C++
globalAnswer = max(globalAnswer, currentAnswer);
```

Question 4: When should I update?

There are three common places.

Before DFS → Preorder

```
C++
process(node);

dfs(left);
dfs(right);
```

Use when processing current node before children.

Between DFS calls → Inorder

```
C++
dfs(left);

process(node);

dfs(right);
```

Usually BST-related problems.

After DFS → Postorder

```
C++
dfs(left);
dfs(right);
```



```
process(node);
```

Use when you need children's results first.

Most **bottom-up DP on trees** uses this.

8. The universal DFS templates

Template 1: Bottom-Up / Return something

```
C++
int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    // use children's answers
    int result = /* calculation */;

    return result;
}
```

Template 2: Bottom-Up + Global Answer

```
C++
int answer = 0;

int dfs(TreeNode* node) {
    if(node == NULL)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    // calculate answer at current node
    answer = max(answer, /* something */);

    // return useful information to parent
    return /* something parent needs */;
}
```

Example:

```
C++
int ans = 0;

int dfs(TreeNode* node) {
    if(!node)
        return 0;

    int left = dfs(node->left);
    int right = dfs(node->right);

    ans = max(ans, left + right);

    return 1 + max(left, right);
}
```

Template 3: Top-Down / Pass state

```
C++
void dfs(TreeNode* node, int state) {
    if(node == NULL)
        return;

    // update state
    state = /* update */;

    dfs(node->left, state);
    dfs(node->right, state);
}
```

9. The most useful visualization

Think of every DFS call as a **small worker**.

For example:

```
C++
dfs(node)
```

The worker says:

"First I'll ask my left child to solve its subtree."

```
C++
int left = dfs(node->left);
```

Then:

"Now I'll ask my right child."

```
C++
int right = dfs(node->right);
```

Then:

"Now I have both answers, so I can solve my own problem."

```
C++
int result = combine(left, right);
```

Finally:

"I'll give my answer to my parent."

```
C++
return result;
```

So the pattern is always:

```
ASK CHILDREN
  ↓
WAIT FOR RESULTS
  ↓
COMBINE RESULTS
  ↓
UPDATE GLOBAL ANSWER IF NEEDED
  ↓
RETURN TO PARENT
```

My recommended approach for every Tree DFS problem

Before touching the code, write this:

1. What does `dfs(node)` represent?
2. What should `NULL` return?
3. What does the parent need from this node?
4. Do I need left and right results first?
5. Is the current node's answer the same thing that should be returned to the parent?
6. If not, use:
 - return value for parent
 - global variable for final answer

The **single most important distinction** to remember is:

```
dfs() RETURN
  =
What my parent needs from me

GLOBAL ANSWER
  =
Best/final answer found anywhere in the tree

PARAMETER
  =
Information passed from parent to child
```

If you remember just this, tree recursion becomes much easier.

ChatGPT can make mistakes. Check important info.